```
// ===== File: Assets/Scripts/Core/GameDatabase.cs =====
using System.Collections.Generic;
using UnityEngine;

/// <summary>
///  资产数据库：集中列出工程内全部可序列化的 SO（道具 / 技能 / 任务），
///  供存档系统按字符串 ID 反查回 ScriptableObject 引用。
///
///  为什么需要它：JSON 不能直接序列化 ScriptableObject 引用，
///  存档只能存 itemSO.id / questSO.questId / skillSO.skillName，
///  读档时必须靠本表把 ID 换回真正的资产对象。
///
///  由编辑器菜单 Tools/Setup/Build Game Database 扫描工程自动生成内容，
///  新增 SO 资产后重跑一次菜单即可。
/// </summary>
[CreateAssetMenu(fileName = "GameDatabase", menuName = "zeldaLike/Game Database")]
public class GameDatabase : ScriptableObject
{
    public List<ItemSO> items = new List<ItemSO>();
    public List<SkillSO> skills = new List<SkillSO>();
    public List<QuestSO> quests = new List<QuestSO>();

    // 运行时索引：资产在编辑器里赋值，字典每次进 Play 重建（不进序列化）
    private Dictionary<string, ItemSO> _itemById;
    private Dictionary<string, SkillSO> _skillByName;
    private Dictionary<string, QuestSO> _questById;

    /// <summary>建立 ID → SO 索引。重复调用是安全的（先清空再重建）。</summary>
    public void BuildIndex()
    {
        _itemById = new Dictionary<string, ItemSO>();
        _skillByName = new Dictionary<string, SkillSO>();
        _questById = new Dictionary<string, QuestSO>();

        foreach (var item in items)
        {
            if (item == null || string.IsNullOrEmpty(item.id)) continue;
            if (!_itemById.ContainsKey(item.id)) _itemById.Add(item.id, item);
                else Debug.LogWarning($"[GameDatabase] 道具 ID 重复：{item.id}，已
        }

        foreach (var skill in skills)
        {
            if (skill == null || string.IsNullOrEmpty(skill.skillName)) continue;
            if (!_skillByName.ContainsKey(skill.skillName)) _skillByName.Add(skill.skillName, skill);
                else Debug.LogWarning($"[GameDatabase] 技能名重复：{skill.skillNa
        }

        foreach (var quest in quests)
        {
```

Copyright (c) 2026 StarEcoStudio

```
            if (quest == null || string.IsNullOrEmpty(quest.questId)) continue;
            if (!_questById.ContainsKey(quest.questId)) _questById.Add(quest.questId, quest);
                    else Debug.LogWarning($"[GameDatabase] 任务 ID 重复：{quest.ques
        }
    }

    public ItemSO GetItem(string id)
    {
        if (_itemById == null) BuildIndex();
        return !string.IsNullOrEmpty(id) && _itemById.TryGetValue(id, out var v) ? v : null;
    }

    public SkillSO GetSkill(string name)
    {
        if (_skillByName == null) BuildIndex();
        return !string.IsNullOrEmpty(name) && _skillByName.TryGetValue(name, out var v) ? v : null;
    }

    public QuestSO GetQuest(string id)
    {
        if (_questById == null) BuildIndex();
        return !string.IsNullOrEmpty(id) && _questById.TryGetValue(id, out var v) ? v : null;
    }
}

// ===== File: Assets/Scripts/Core/GameInput.cs =====
using UnityEngine;
using UnityEngine.InputSystem;

/// <summary>
///  输入抽象门面（动作层）：游戏逻辑只读「动作」（Move / Attack / Interact…），
///  不直接读物理按键。PC 键鼠、手柄、手机触屏各自把物理输入绑定到同一套动作，
///  加端 / 换端只需补绑定，不改任何游戏逻辑。
///  基于 Unity Input System（com.unity.inputsystem），键位与旧 InputManager 一致：
///  WASD / 方向键 = 移动，K = 挥剑，J = 射箭，F = 交互，Q = 切换武器，1 / 2 / L = 面板，Shift = 冲刺，Esc = 取消。
///  手柄 / 触屏绑定为 OnScreenStick / OnScreenButton 等虚拟控件预留（写入 Gamepad 控件位）。
/// </summary>
public static class GameInput
{
    private static readonly InputAction move;
    private static readonly InputAction sprint;
    private static readonly InputAction attack;
    private static readonly InputAction shoot;
    private static readonly InputAction interact;
    private static readonly InputAction changeEquipment;
    private static readonly InputAction toggleStats;
    private static readonly InputAction toggleSkillTree;
    private static readonly InputAction toggleQuestLog;
    private static readonly InputAction cancel;

    static GameInput()
```

Copyright (c) 2026 StarEcoStudio

```
    {
            //  移动：ＷＡＳＤ  +  方向键  +  手柄左摇杆（虚拟摇杆通过  Ｇａｍｅｐａｄ  左摇杆位写入）
        move = new InputAction("Move", InputActionType.Value, expectedControlType: "Vector2");
        move.AddCompositeBinding("2DVector")
            .With("Up", "<Keyboard>/w")
            .With("Down", "<Keyboard>/s")
            .With("Left", "<Keyboard>/a")
            .With("Right", "<Keyboard>/d");
        move.AddCompositeBinding("2DVector")
            .With("Up", "<Keyboard>/upArrow")
            .With("Down", "<Keyboard>/downArrow")
            .With("Left", "<Keyboard>/leftArrow")
            .With("Right", "<Keyboard>/rightArrow");
        move.AddBinding("<Gamepad>/leftStick");

        sprint = NewButton("Sprint", "<Keyboard>/leftShift", "<Gamepad>/leftShoulder");
        attack = NewButton("Attack", "<Keyboard>/k", "<Gamepad>/buttonSouth");
        shoot = NewButton("Shoot", "<Keyboard>/j", "<Gamepad>/buttonWest");
        interact = NewButton("Interact", "<Keyboard>/f", "<Gamepad>/buttonNorth");
        changeEquipment = NewButton("ChangeEquipment", "<Keyboard>/q", "<Gamepad>/rightShoulder");
        toggleStats = NewButton("ToggleStats", "<Keyboard>/1");
        toggleSkillTree = NewButton("ToggleSkillTree", "<Keyboard>/2");
        toggleQuestLog = NewButton("ToggleQuestLog", "<Keyboard>/l");
        cancel = NewButton("Cancel", "<Keyboard>/escape", "<Gamepad>/select");

        move.Enable();
        sprint.Enable();
        attack.Enable();
        shoot.Enable();
        interact.Enable();
        changeEquipment.Enable();
        toggleStats.Enable();
        toggleSkillTree.Enable();
        toggleQuestLog.Enable();
        cancel.Enable();
    }

    private static InputAction NewButton(string name, params string[] bindings)
    {
        var action = new InputAction(name, InputActionType.Button);
        foreach (string path in bindings)
            action.AddBinding(path);
        return action;
    }

       /// <summary>移动向量（2DVector 合成控件，已归一化）。</summary>
    public static Vector2 Move => move.ReadValue<Vector2>();

       /// <summary>冲刺是否按住。</summary>
    public static bool SprintHeld => sprint.IsPressed();
```

Copyright (c) 2026 StarEcoStudio

```
        /// <summary>本帧按下挥剑。</summary>
    public static bool AttackPressed => attack.WasPressedThisFrame();

        /// <summary>本帧按下射箭。</summary>
    public static bool ShootPressed => shoot.WasPressedThisFrame();

        /// <summary>本帧按下交互。</summary>
    public static bool InteractPressed => interact.WasPressedThisFrame();

        /// <summary>本帧按下切换武器。</summary>
    public static bool ChangeEquipmentPressed => changeEquipment.WasPressedThisFrame();

        /// <summary>本帧按下属性面板开关。</summary>
    public static bool ToggleStatsPressed => toggleStats.WasPressedThisFrame();

        /// <summary>本帧按下技能树面板开关。</summary>
    public static bool ToggleSkillTreePressed => toggleSkillTree.WasPressedThisFrame();

        /// <summary>本帧按下任务日志面板开关。</summary>
    public static bool ToggleQuestLogPressed => toggleQuestLog.WasPressedThisFrame();

        /// <summary>本帧按下取消（Esc，关商店等）。</summary>
    public static bool CancelPressed => cancel.WasPressedThisFrame();
}

// ===== File: Assets/Scripts/Core/SaveData.cs =====
using System;
using System.Collections.Generic;

/// <summary>
/// 存档数据结构（纯数据，不含任何 Unity 对象引用）。
///
/// JsonUtility 的两条硬约束决定了这里的形状：
/// 1. 不能序列化 Dictionary —— 所以任务状态 / 进度用「键 + 值」的平行列表表达；
/// 2. 不能序列化 ScriptableObject 引用 —— 所以道具 / 技能 / 任务只存字符串 ID，
///       读档时由 GameDatabase 反查回资产对象。
/// </summary>
[Serializable]
public class SaveData
{
    public const string CurrentVersion = "1.0";

    public string version = CurrentVersion;
    public string savedAtIso;

        /// <summary>存档时的场景名与玩家坐标，用于读档后回到原位置。</summary>
    public string sceneName;
    public float playerX;
    public float playerY;
```

Copyright (c) 2026 StarEcoStudio

```
    public StatsData stats = new StatsData();
    public InventoryData inventory = new InventoryData();
    public SkillData skills = new SkillData();
    public ExpData exp = new ExpData();
    public QuestsData quests = new QuestsData();
}

[Serializable]
public class StatsData
{
    public int playerDamage;
    public int playerSpeed;
    public float playerMaxHealth;
    public float playerCurrentHealth;
    public float playerweaponRange;
    public float playerKnockbackForce;
    public float playerKnockbackStun;
    public bool hasSword;
    public bool hasBow;
}

[Serializable]
public class InventoryData
{
    public int gold;
       /// <summary>每项对应一个背包槽（按索引对齐 InventoryManager.itemSlots）。</summary>
    public List<InventorySlotData> slots = new List<InventorySlotData>();
}

[Serializable]
public class InventorySlotData
{
       /// <summary>ItemSO.id；空槽为空串。</summary>
    public string itemId;
    public int quantity;
}

[Serializable]
public class SkillData
{
    public int availablePoints;
    public List<SkillSlotData> slots = new List<SkillSlotData>();
}

[Serializable]
public class SkillSlotData
{
       /// <summary>SkillSO.skillName；空槽为空串。</summary>
    public string skillName;
```

Copyright (c) 2026 StarEcoStudio

```
    public int currentLevel;
    public bool isUnlocked;
}

[Serializable]
public class ExpData
{
    public int level;
    public int currentExp;
    public int expToLevel;
    public float expGrowMultiplier;
}

/// <summary>任务状态与进度。QuestState 本身是 enum，JsonUtility 不支持字典，
/// 这里存枚举的整数值，由 QuestManager 转回。</summary>
[Serializable]
public class QuestsData
{
    /// <summary>已接取 / 进行中 / 已完成 / 已交付的任务 ID 列表。</summary>
    public List<string> activeQuestIds = new List<string>();

    /// <summary>questId → 状态枚举值。</summary>
    public List<QuestStateEntry> states = new List<QuestStateEntry>();

    /// <summary>questId → 该任务各目标的进度数组。</summary>
    public List<QuestProgressEntry> progress = new List<QuestProgressEntry>();
}

[Serializable]
public class QuestStateEntry
{
    public string questId;
    public int stateValue;
}

[Serializable]
public class QuestProgressEntry
{
    public string questId;
    public int[] objectiveProgress;
}

// ===== File: Assets/Scripts/Core/SaveSystem.cs =====
using System;
using System.Collections.Generic;
using System.IO;
using UnityEngine;
using UnityEngine.SceneManagement;

/// <summary>
public class SaveSystem : MonoBehaviour
```

Copyright (c) 2026 StarEcoStudio

```
{
    [Ｔｏｏｌｔｉｐ("道具/技能/任务资产库，由 Ｔｏｏｌｓ/Ｓｅｔｕｐ/Ｂｕｉｌｄ Ｇａｍｅ Ｄａｔａｂａｓｅ 生成")]
    public GameDatabase database;

    public static SaveSystem Instance { get; private set; }

    private const string FileName = "save.json";

    ///  <ｓｕｍｍａｒｙ>读档时若需要先切场景，数据挂在这里，等场景加载完再落地。</ｓｕｍｍａｒｙ>
    private SaveData _pendingRestore;

    public static string SavePath => Path.Combine(Application.persistentDataPath, FileName);

    public static bool HasSave => File.Exists(SavePath);

    void Awake()
    {
        if (Instance != null && Instance != this)
        {
            Destroy(this);
            return;
        }
        Instance = this;
        database?.BuildIndex();
    }

    void OnEnable()
    {
        SceneManager.sceneLoaded += OnSceneLoaded;
    }

    void OnDisable()
    {
        SceneManager.sceneLoaded -= OnSceneLoaded;
    }

    ///  <ｓｕｍｍａｒｙ>退出游戏前自动存档，避免玩家忘记手动存。</ｓｕｍｍａｒｙ>
    void OnApplicationQuit()
    {
        SaveGame();
    }

    private void OnSceneLoaded(Scene scene, LoadSceneMode mode)
    {
            //  读档触发的切场景：等场景里各管理器 Ｓｔａｒｔ 跑完再应用数据
        if (_pendingRestore == null) return;
        ApplyData(_pendingRestore);
        _pendingRestore = null;
    }

    public bool SaveGame()
```

Copyright (c) 2026 StarEcoStudio

```
    {
        var data = CaptureData();
        if (data == null)
        {
                    Ｄｅｂｕｇ．ＬｏｇＷａｒｎｉｎｇ（"［ＳａｖｅＳｙｓｔｅｍ］ 存档失败：关键管理器未就绪"）；
            return false;
        }

        try
        {
            File.WriteAllText(SavePath, JsonUtility.ToJson(data, true));
                    Ｄｅｂｕｇ．Ｌｏｇ（$"［ＳａｖｅＳｙｓｔｅｍ］ 已存档 → ｛ＳａｖｅＰａｔｈ｝"）；
            return true;
        }
        catch (Exception e)
        {
                    Ｄｅｂｕｇ．ＬｏｇＥｒｒｏｒ（$"［ＳａｖｅＳｙｓｔｅｍ］ 写档失败：｛ｅ．Ｍｅｓｓａｇｅ｝"）；
            return false;
        }
    }

    /// ＜ｓｕｍｍａｒｙ＞读档。若存档场景与当前不同会先切场景，返回是否成功。＜／ｓｕｍｍａｒｙ＞
    public bool LoadGame()
    {
        var data = ReadData();
        if (data == null) return false;

        if (!string.IsNullOrEmpty(data.sceneName)
            && data.sceneName != SceneManager.GetActiveScene().name
            && Application.CanStreamedLevelBeLoaded(data.sceneName))
        {
            _pendingRestore = data;
            SceneManager.LoadScene(data.sceneName);
            return true;
        }

        ApplyData(data);
        return true;
    }

    /// ＜ｓｕｍｍａｒｙ＞删除存档文件（调试与「重新开始」用）。＜／ｓｕｍｍａｒｙ＞
    public void DeleteSave()
    {
        if (File.Exists(SavePath)) File.Delete(SavePath);
              Ｄｅｂｕｇ．Ｌｏｇ（"［ＳａｖｅＳｙｓｔｅｍ］ 存档已删除"）；
    }

    // ＝＝＝＝＝＝＝＝＝＝ 读盘 ＝＝＝＝＝＝＝＝＝＝

    public SaveData ReadData()
```

Copyright (c) 2026 StarEcoStudio

```
    {
        if (!File.Exists(SavePath))
        {
                    Ｄｅｂｕｇ．Ｌｏｇ（"［ＳａｖｅＳｙｓｔｅｍ］ 没有存档文件"）；
            return null;
        }

        try
        {
            var data = JsonUtility.FromJson<SaveData>(File.ReadAllText(SavePath));
            if (data == null)
            {
                        Ｄｅｂｕｇ．ＬｏｇＥｒｒｏｒ（"［ＳａｖｅＳｙｓｔｅｍ］ 存档解析失败"）；
                return null;
            }
                    Ｄｅｂｕｇ．Ｌｏｇ（＄"［ＳａｖｅＳｙｓｔｅｍ］ 已读取存档（版本 ｛ｄａｔａ．ｖｅｒｓｉｏｎ｝，存于 ｛ｄａｔａ．ｓ
            return data;
        }
        catch (Exception e)
        {
                    Ｄｅｂｕｇ．ＬｏｇＥｒｒｏｒ（＄"［ＳａｖｅＳｙｓｔｅｍ］ 读档失败：｛ｅ．Ｍｅｓｓａｇｅ｝"）；
            return null;
        }
    }

       //  ＝＝＝＝＝＝＝＝＝＝ 采集：游戏状态 → ＳａｖｅＤａｔａ ＝＝＝＝＝＝＝＝＝＝

    private SaveData CaptureData()
    {
        var stats = StatsManager.Instance;
        var inv = InventoryManager.Instance;
        if (stats == null || inv == null) return null;

        var data = new SaveData
        {
            savedAtIso = DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"),
            sceneName = SceneManager.GetActiveScene().name
        };

        var player = FindPlayer();
        if (player != null)
        {
            var pos = GetPlayerPosition(player);
            data.playerX = pos.x;
            data.playerY = pos.y;
        }

        CaptureStats(stats, data);
        CaptureInventory(inv, data);
        CaptureExp(data);
```

Copyright (c) 2026 StarEcoStudio

```
        var skills = FindSkillTreeManager();
        if (skills != null) CaptureSkills(skills, data);

        var quest = QuestManager.Instance;
        if (quest != null) CaptureQuests(quest, data);

        return data;
    }

    private static void CaptureStats(StatsManager s, SaveData data)
    {
        data.stats = new StatsData
        {
            playerDamage = s.playerDamage,
            playerSpeed = s.playerSpeed,
            playerMaxHealth = s.playerMaxHealth,
            playerCurrentHealth = s.playerCurrentHealth,
            playerweaponRange = s.playerweaponRange,
            playerKnockbackForce = s.playerKnockbackForce,
            playerKnockbackStun = s.playerKnockbackStun,
            hasSword = s.hasSword,
            hasBow = s.hasBow
        };
    }

    private static void CaptureInventory(InventoryManager inv, SaveData data)
    {
        data.inventory = new InventoryData { gold = inv.gold };
        if (inv.itemSlots == null) return;

        foreach (var slot in inv.itemSlots)
        {
            data.inventory.slots.Add(new InventorySlotData
            {
                itemId = slot.itemSO != null ? slot.itemSO.id : string.Empty,
                quantity = slot.quantity
            });
        }
    }

    private static void CaptureExp(SaveData data)
    {
        var exp = ExpManager.Instance;
        if (exp == null) return;

        data.exp = new ExpData
        {
            level = exp.level,
            currentExp = exp.currentExp,
```

Copyright (c) 2026 StarEcoStudio

```
            expToLevel = exp.expToLevel,
            expGrowMultiplier = exp.expGrowMultiplier
        };
    }

    private static void CaptureSkills(SkillTreeManager tree, SaveData data)
    {
        data.skills = new SkillData { availablePoints = tree.availablePoints };
        if (tree.skillSlots == null) return;

        foreach (var slot in tree.skillSlots)
        {
            data.skills.slots.Add(new SkillSlotData
            {
                skillName = slot.skillSO != null ? slot.skillSO.skillName : string.Empty,
                currentLevel = slot.currentLevel,
                isUnlocked = slot.isUnlocked
            });
        }
    }

    private static void CaptureQuests(QuestManager quest, SaveData data)
    {
        data.quests = quest.ExportSaveData();
    }

    // ========== 应用：SaveData → 游戏状态 ==========

    private void ApplyData(SaveData data)
    {
        if (database == null)
        {
                Debug.LogError("[SaveSystem] 未配置 GameDatabase，无法按 ID 还原资产。"
                                "请执行菜单 Tools/Setup/Build Game Database 并把生成的
            return;
        }
        database.BuildIndex();

        ApplyPlayerPosition(data);
        ApplyStats(data.stats);
        ApplyInventory(data.inventory);
        ApplyExp(data.exp);
        ApplySkills(data.skills);
        ApplyQuests(data.quests);

            Debug.Log("[SaveSystem] 读档完成");
    }

    private static void ApplyPlayerPosition(SaveData data)
    {
```

Copyright (c) 2026 StarEcoStudio

```
        var player = FindPlayer();
        if (player == null) return;

              //  玩家是刚体：直接写 position，避免用 velocity 被状态机当帧清零
        var body = player.GetComponent<Rigidbody2D>();
        if (body != null)
        {
            body.position = new Vector2(data.playerX, data.playerY);
                    //  清零速度与角速度，否则跨场景读档时旧场景残留的速度会让玩家
                    //  从存档点继续漂移，导致「读档后位置对不上」。
            body.velocity = Vector2.zero;
            body.angularVelocity = 0f;
        }
        else player.transform.position = new Vector2(data.playerX, data.playerY);
    }

    /// <summary>
    ///  读取玩家世界坐标。与 <see cref="ApplyPlayerPosition" /> 保持同一口径：
    ///  有刚体时以 Rigidbody2D.position 为准，否则回退 transform.position。
    /// </summary>
    private static Vector3 GetPlayerPosition(PlayerController player)
    {
        var body = player.GetComponent<Rigidbody2D>();
        return body != null ? (Vector3)body.position : player.transform.position;
    }

    private static void ApplyStats(StatsData s)
    {
        var stats = StatsManager.Instance;
        if (stats == null || s == null) return;

        stats.playerDamage = s.playerDamage;
        stats.playerSpeed = s.playerSpeed;
        stats.playerMaxHealth = s.playerMaxHealth;
        stats.playerCurrentHealth = s.playerCurrentHealth;
        stats.playerweaponRange = s.playerweaponRange;
        stats.playerKnockbackForce = s.playerKnockbackForce;
        stats.playerKnockbackStun = s.playerKnockbackStun;
        stats.hasSword = s.hasSword;
        stats.hasBow = s.hasBow;

        if (stats.playerHealthText != null)
            stats.playerHealthText.text = "HP:" + stats.playerCurrentHealth + "/" + stats.playerMaxHealth;
        if (stats.statsUI != null) stats.statsUI.UpdateAllStats();
    }

    private void ApplyInventory(InventoryData inv)
    {
        var manager = InventoryManager.Instance;
        if (manager == null || inv == null) return;
```

Copyright (c) 2026 StarEcoStudio

```
        manager.gold = inv.gold;
        if (manager.goldText != null) manager.goldText.text = manager.gold.ToString();

        if (manager.itemSlots == null) return;

        for (int i = 0; i < manager.itemSlots.Length && i < inv.slots.Count; i++)
        {
            var slot = manager.itemSlots[i];
            var saved = inv.slots[i];

            slot.itemSO = string.IsNullOrEmpty(saved.itemId) ? null : database.GetItem(saved.itemId);
            if (slot.itemSO == null && !string.IsNullOrEmpty(saved.itemId))
                        Debug.LogWarning($"[SaveSystem] 道具 ID 在 GameDatabase 中不

            slot.quantity = slot.itemSO != null ? saved.quantity : 0;
            slot.UpdateUI();
        }
    }

    private static void ApplyExp(ExpData e)
    {
        var exp = ExpManager.Instance;
        if (exp == null || e == null) return;

        exp.level = e.level;
        exp.currentExp = e.currentExp;
        exp.expToLevel = e.expToLevel;
        exp.expGrowMultiplier = e.expGrowMultiplier;
        exp.UpdateUI();
    }

    private void ApplySkills(SkillData s)
    {
        var tree = FindSkillTreeManager();
        if (tree == null || s == null) return;

        tree.availablePoints = s.availablePoints;
        if (tree.skillSlots != null)
        {
            for (int i = 0; i < tree.skillSlots.Length && i < s.slots.Count; i++)
            {
                var slot = tree.skillSlots[i];
                var saved = s.slots[i];

                slot.skillSO = string.IsNullOrEmpty(saved.skillName) ? null : database.GetSkill(saved.skillName)
                if (slot.skillSO == null && !string.IsNullOrEmpty(saved.skillName))
                            Debug.LogWarning($"[SaveSystem] 技能名在 GameDatabase 中

                slot.currentLevel = saved.currentLevel;
```

Copyright (c) 2026 StarEcoStudio

```
                slot.isUnlocked = saved.isUnlocked;
                slot.RefreshUI();
            }
        }
             tree.UpdateAbilityPoints(0);  //  顺带刷新点数文本

             //  注意：这里**不能**按技能等级重放属性效果。
             //  stats 段存的是「已含技能加成」的最终属性（存档发生在花点之后），
             //  若再叠加一次技能增量就会双重计算（实测 maxHP 102→104、dmg 4→7）。
             //  技能效果与属性基线由 stats 段一次性对齐，技能等级只负责展示与后续加点。
    }

    private void ApplyQuests(QuestsData q)
    {
        var quest = QuestManager.Instance;
        if (quest == null || q == null) return;

        quest.ImportSaveData(q, database);
    }

       //  ========== 查找辅助 ==========

    private static PlayerController FindPlayer()
    {
             //  玩家随 GameRoot 持久存活，场景里可能同时存在已停用的旧实例，取活动的那个
        return FindObjectOfType<PlayerController>();
    }

    private static SkillTreeManager FindSkillTreeManager()
    {
        return FindObjectOfType<SkillTreeManager>();
    }
}

// ===== File: Assets/Scripts/Enemy/EnemyController.cs =====
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class EnemyController : MonoBehaviour
{
    public float damageAmount = 25f;
    public float attackRange = 2;
    public float knockbackForce = 5f;
    public Transform player;
    public Animator anim;
    public Rigidbody2D rb;
    public bool canChase = true;
    public EnemyMovement en;

       [Header("巡视")]
```

Copyright (c) 2026 StarEcoStudio

```
    public Transform[] patrolPoints;
    public float patrolSpeed = 1.5f;
    public float patrolWaitTime = 1.5f;
    [HideInInspector] public int patrolIndex;
    [HideInInspector] public float patrolWait;

    private StateMachine<EnemyController> enemyStateMachine;
    private EnemyState currentStateEnum;

    void Start()
    {
        enemyStateMachine = new StateMachine<EnemyController>(this);
              // 配了路点就从巡视开始，否则保持原站桩
        ChangeState(patrolPoints != null && patrolPoints.Length >= 2
            ? EnemyState.Patrolling
            : EnemyState.Idle);
    }

    public void ChangeState(EnemyState newState)
    {
        currentStateEnum = newState;
        IEnemyState next = newState switch
        {
            EnemyState.Idle => IdleState.Instance,
            EnemyState.Patrolling => PatrolState.Instance,
            EnemyState.Chasing => ChaseState.Instance,
            EnemyState.Attacking => AttackState.Instance,
            _ => IdleState.Instance
        };
        enemyStateMachine.ChangeState(next);
    }

    public void Attack()
    {
        if (player == null || !player.gameObject.activeInHierarchy)
            return;

        UnityEngine.Debug.Log("Attacking player now!");
        if (Vector2.Distance(player.position, transform.position) <= attackRange)
        {
            player.gameObject.GetComponent<PlayerHealth>().ChangeHealth(-damageAmount);

                    // 本次伤害若已致死，PlayerHealth 会停用对象，不再施加击退与硬直
            if (!player.gameObject.activeInHierarchy)
                return;

            player.GetComponent<PlayerKnockbackable>().KnockBack(transform, knockbackForce);
            player.GetComponent<PlayerController>().ChangeState(PlayerKnockbackState.Instance);
        }
    }
```

Copyright (c) 2026 StarEcoStudio

```
    /// <summary>
       /// 攻击动画收尾（由 Enemy_Torch_Attacking.anim 的动画事件调用）。
       /// 配了路点回巡视，否则回站桩。
    /// </summary>
    public void OnAttackAnimComplete()
    {
        ChangeState(patrolPoints != null && patrolPoints.Length >= 2
            ? EnemyState.Patrolling
            : EnemyState.Idle);
    }

    public void StartAttackCooldown(float duration)
    {
        StartCoroutine(AttackCooldownRoutine(duration));
    }

    private IEnumerator AttackCooldownRoutine(float duration)
    {
        yield return new WaitForSeconds(duration);
        canChase = true;

              // 玩家仍存活且未走出侦测圈（状态未被退出事件切走）时，恢复追击
        if (currentStateEnum == EnemyState.Attacking
            && player != null && player.gameObject.activeInHierarchy)
        {
            ChangeState(EnemyState.Chasing);
        }
    }

    void Update()
    {
        enemyStateMachine.Tick();
    }
}

// ===== File: Assets/Scripts/Enemy/EnemyHealth.cs =====
using UnityEngine;

public class EnemyHealth : MonoBehaviour
{
       [Tooltip("敌人类型标识（全小写英文，如 torch / slime），供任务系统击杀目标匹配")]
    public string enemyId = "enemy";

    public int expReward = 3;
    public delegate void MonsterDefeated(string enemyId, int exp);
    public static event MonsterDefeated OnMonsterDefeated;
    private void Start()
    {
        StatsManager.Instance.enemyCurrentHealth = StatsManager.Instance.enemyMaxHealth;
```

Copyright (c) 2026 StarEcoStudio

```
    }
    public void ChangeHealth(int amount)
    {
        StatsManager.Instance.enemyCurrentHealth += amount;
        if (StatsManager.Instance.enemyCurrentHealth > StatsManager.Instance.enemyMaxHealth)
        {
            StatsManager.Instance.enemyCurrentHealth = StatsManager.Instance.enemyMaxHealth;
        }
        else if (StatsManager.Instance.enemyCurrentHealth <= 0)
        {
            Destroy(gameObject);
            OnMonsterDefeated(enemyId, expReward);
        }
    }
}

// ===== File: Assets/Scripts/Enemy/EnemyKnockbackable.cs =====
using System.Collections;
using UnityEngine;

public class EnemyKnockbackable : MonoBehaviour
{
    private Rigidbody2D rb;
    public bool isKnockback = false;
    private void Start()
    {
        rb = GetComponent<Rigidbody2D>();
    }
    public void Knockback(Transform forceTransform, float playerKnockbackForce, float stunDuration)
    {
        isKnockback = true;
        Vector2 direction = (transform.position - forceTransform.position).normalized;
        rb.velocity = direction * playerKnockbackForce;
        StartCoroutine(KnockbackCoolDown(stunDuration));
    }
    public IEnumerator KnockbackCoolDown(float duration)
    {
        yield return new WaitForSeconds(duration);
        isKnockback = false;
    }
}

// ===== File: Assets/Scripts/Enemy/EnemyMovement.cs =====
using System.Collections;
using UnityEngine;

public class EnemyMovement : MonoBehaviour
{

    public float speed;
```

Copyright (c) 2026 StarEcoStudio

```
    public int facingDirection = 1;
    public EnemyController co;
    [UnityEngine.Serialization.FormerlySerializedAs("enemy_Knockback")]
    public EnemyKnockbackable enemyKnockbackable;

    void Flip()
    {
        facingDirection *= -1;
        transform.localScale = new Vector3(transform.localScale.x * -1, transform.localScale.y, transform.localS
    }


    public void Chase()
    {
        if (enemyKnockbackable.isKnockback == false)
        {
            if (co.player.position.x > transform.position.x && facingDirection == -1
                    || co.player.position.x < transform.position.x && facingDirection == 1)
            {
                Flip();
            }

            Vector2 direction = (co.player.position - transform.position).normalized;
            co.rb.velocity = direction * speed;
        }

    }

    public void Patrol(Vector2 target)
    {
        if (enemyKnockbackable.isKnockback == false)
        {
            if (target.x > transform.position.x && facingDirection == -1
                    || target.x < transform.position.x && facingDirection == 1)
            {
                Flip();
            }

            Vector2 direction = (target - (Vector2)transform.position).normalized;
            co.rb.velocity = direction * co.patrolSpeed;
        }
    }
}
public enum EnemyState
{
    Idle,
    Patrolling,
    Chasing,
    Attacking,

}
```

Copyright (c) 2026 StarEcoStudio

```
// ===== File: Assets/Scripts/Enemy/EnemyPlayerDetector.cs =====
using UnityEngine;

public class EnemyPlayerDetector : MonoBehaviour
{
    public EnemyController enemyController;

    private void OnTriggerStay2D(Collider2D collision)
    {
        if (collision.gameObject.tag == "Player" && enemyController.canChase)
        {
            if (enemyController.player == null)
            {
                enemyController.player = collision.transform;
            }

            enemyController.ChangeState(EnemyState.Chasing);
        }
    }
    private void OnTriggerExit2D(Collider2D collision)
    {

        if (collision.gameObject.tag == "Player")
        {
                    // 敌人被销毁的同帧仍可能补发退出事件，判空避免 MissingReferenceException
            if (enemyController != null && enemyController.rb != null)
            {
                enemyController.rb.velocity = Vector2.zero;
            }

            if (enemyController != null)
            {
                        // 配了路点就回巡视，否则回站桩
                enemyController.ChangeState(
                    enemyController.patrolPoints != null && enemyController.patrolPoints.Length >= 2
                        ? EnemyState.Patrolling
                        : EnemyState.Idle);
            }
        }

    }

}

// ===== File: Assets/Scripts/Enemy/States/AttackState.cs =====
using UnityEngine;

public class AttackState : IEnemyState
{
```

Copyright (c) 2026 StarEcoStudio

```
    public static readonly AttackState Instance = new AttackState();

    public void OnEnter(EnemyController e)
    {
        e.anim.SetBool("isAttacking", true);
    }

    public void OnUpdate(EnemyController e)
    {
    }

    public void OnExit(EnemyController e)
    {
        e.anim.SetBool("isAttacking", false);
    }
}

// ===== File: Assets/Scripts/Enemy/States/ChaseState.cs =====
using UnityEngine;

public class ChaseState : IEnemyState
{
    public static readonly ChaseState Instance = new ChaseState();

    public void OnEnter(EnemyController e)
    {
        e.anim.SetBool("isChasing", true);
    }

    public void OnUpdate(EnemyController e)
    {
        e.en.Chase();

        if (Vector2.Distance(e.transform.position, e.player.position) <= e.attackRange)
        {
            e.rb.velocity = Vector2.zero;
            e.canChase = false;
            e.ChangeState(EnemyState.Attacking);
            e.StartAttackCooldown(2f);
        }
    }

    public void OnExit(EnemyController e)
    {
        e.anim.SetBool("isChasing", false);
    }
}

// ===== File: Assets/Scripts/Enemy/States/IEnemyState.cs =====
public interface IEnemyState : IState<EnemyController> { }
```

Copyright (c) 2026 StarEcoStudio

```
// ===== File: Assets/Scripts/Enemy/States/IdleState.cs =====
using UnityEngine;

public class IdleState : IEnemyState
{
    public static readonly IdleState Instance = new IdleState();

    public void OnEnter(EnemyController e)
    {
        e.anim.SetBool("isIdle", true);
    }

    public void OnUpdate(EnemyController e)
    {
    }

    public void OnExit(EnemyController e)
    {
        e.anim.SetBool("isIdle", false);
    }
}

// ===== File: Assets/Scripts/Enemy/States/PatrolState.cs =====
using UnityEngine;

public class PatrolState : IEnemyState
{
    public static readonly PatrolState Instance = new PatrolState();

    private const float ReachDistance = 0.15f;

    public void OnEnter(EnemyController e)
    {
              / /  路点不足两个视为未配置巡视，回退站桩，避免空引用死循环
        if (e.patrolPoints == null || e.patrolPoints.Length < 2)
        {
            e.ChangeState(EnemyState.Idle);
            return;
        }

        e.anim.SetBool("isIdle", false);
    }

    public void OnUpdate(EnemyController e)
    {
        if (e.patrolPoints == null || e.patrolPoints.Length < 2)
        {
            e.ChangeState(EnemyState.Idle);
            return;
```

Copyright (c) 2026 StarEcoStudio

```
        }

              // 到点停留：原地等待，切待机动画
        if (e.patrolWait > 0f)
        {
            e.patrolWait -= Time.deltaTime;
            e.rb.velocity = Vector2.zero;
            e.anim.SetBool("isIdle", true);
            return;
        }

        Transform target = e.patrolPoints[e.patrolIndex];
        if (target == null)
        {
                    // 路点被删则跳过，指向下一个
            e.patrolIndex = (e.patrolIndex + 1) % e.patrolPoints.Length;
            return;
        }

        e.anim.SetBool("isIdle", false);
        e.en.Patrol(target.position);

        if (Vector2.Distance(e.transform.position, target.position) <= ReachDistance)
        {
            e.rb.velocity = Vector2.zero;
            e.patrolIndex = (e.patrolIndex + 1) % e.patrolPoints.Length;
            e.patrolWait = e.patrolWaitTime;
        }
    }

    public void OnExit(EnemyController e)
    {
        if (e.rb != null)
        {
            e.rb.velocity = Vector2.zero;
        }

        e.anim.SetBool("isIdle", false);
    }
}

// ===== File: Assets/Scripts/Inventory & Shop/ItemSO.cs =====
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// 道具可解锁的战斗能力。遵循塞尔达 " 能力即钥匙 " 设计：
/// 战斗能力不是开局默认拥有，而是通过对应道具永久解锁。
/// </summary>
public enum Ability
```

Copyright (c) 2026 StarEcoStudio

```
{
    None,
    Sword,
    Bow
}

[CreateAssetMenu(fileName ="New Item")]
public class ItemSO : ScriptableObject
{
       [Ｔｏｏｌｔｉｐ（"内部标识：全小写英文，用于代码查找与场景物体命名（如 Ｌｏｏｔ＿ｓｗｏｒｄ），不展示给玩家"）]
    public string id;

       [Ｔｏｏｌｔｉｐ（"显示名：给玩家看的文本，可使用中文"）]
    public string itemName;

    [TextArea] public string itemDescription;
    public Sprite icon;
    public bool isGold;
    public int stackSize = 3;

    [Header("Stats")]
    public int currentHealth;
    public int maxHealth;
    public int speed;
    public int damage;

    [Header("For Temporary Items")]
    public float duration;

    [Header("Ability Unlock")]
       [Ｔｏｏｌｔｉｐ（"使用该道具后永久解锁的战斗能力，与临时 ｂｕｆｆ 的 ｄｕｒａｔｉｏｎ 机制相互独立"）]
    public Ability unlocksAbility = Ability.None;

    [Header("World Presentation")]
       [Ｔｏｏｌｔｉｐ（"地面拾取物的逐帧动画序列（精灵图集切割而来）；留空时地面直接显示 ｉｃｏｎ 单图"）]
    public Sprite[] worldFrames;
       [Ｔｏｏｌｔｉｐ（"地面帧动画播放速率（帧/秒）"）]
    public float worldFrameRate = 8f;

}

// ===== File: Assets/Scripts/Inventory & Shop/Loot.cs =====
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class Loot : MonoBehaviour
{
    public ItemSO itemSO;
```

Copyright (c) 2026 StarEcoStudio

```
    public SpriteRenderer sr;
    public Animator anim;
    public static event Action<ItemSO, int> OnItemLooted;

    public int quantity;
    private bool canPickup = true;
    private bool playerInside = false;

       //  地面逐帧动画状态（精灵图集帧序列；没有配 ｗｏｒｌｄＦｒａｍｅｓ 的道具仍用 ｉｃｏｎ 单图）
    private Sprite[] worldFrames;
    private float worldFrameRate = 8f;
    private int frameIndex;
    private float frameTimer;
    private bool pickedUp;

       //  地面掉落物的最大世界尺寸（长边）。不同来源的图标像素 / ＰＰＵ 差异很大
       //  （例如 ＡＩ 生成的 １０２４ｐｘ 图默认会显示成 １０ 个世界单位），统一归一化避免尺寸失控
    private const float MaxWorldSize = 0.8f;
    private const float BaseColliderRadius = 0.5f;

    private void Start()
    {
        NormalizeWorldSize();
    }

    private void OnValidate()
    {
        if (itemSO == null)
            return;

        UpdateAppearance();

    }
    private void UpdateAppearance()
    {
              //  地面优先播放精灵图集帧动画；普通道具没有配帧序列时回退到背包 ｉｃｏｎ 单图
        worldFrames = itemSO.worldFrames != null && itemSO.worldFrames.Length > 0
            ? itemSO.worldFrames : null;
        worldFrameRate = itemSO.worldFrameRate;
        frameIndex = 0;
        frameTimer = 0f;
        sr.sprite = worldFrames != null ? worldFrames[0] : itemSO.icon;
              //  层级面板用英文内部名（Ｌｏｏｔ＿ｓｗｏｒｄ / Ｌｏｏｔ＿ｂｏｗ ．．．）；中文 ｉｔｅｍＮａｍｅ 只用于 ＵＩ 显示。
              //  ｉｄ 未配置时回退到资产文件名（也是英文），保证层级永远不出现中文物体名。
        string id = string.IsNullOrWhiteSpace(itemSO.id) ? itemSO.name : itemSO.id;
        this.name = "Loot_" + id;
        NormalizeWorldSize();
    }

       //  过大的图标按比例缩小根物体；正常尺寸的素材保持 １：１。幂等，可重复调用。
    private void NormalizeWorldSize()
```

Copyright (c) 2026 StarEcoStudio

```
    {
        if (sr == null || sr.sprite == null) return;

        transform.localScale = Vector3.one;
              //  像素扫描只用于图集切割帧（图集已开启 Ｒｅａｄ/Ｗｒｉｔｅ）；
              //  普通道具的 ｉｃｏｎ 纹理不可读，直接用精灵包围盒，避免抛 ＧｅｔＰｉｘｅｌｓ 异常
        float spriteSize = worldFrames != null
            ? GetSpriteContentSize(sr.sprite)
            : Mathf.Max(sr.sprite.bounds.size.x, sr.sprite.bounds.size.y);
        float factor = spriteSize > MaxWorldSize ? MaxWorldSize / spriteSize : 1f;
        transform.localScale = Vector3.one * factor;

              //  补偿碰撞器：根物体缩小后保持世界中的拾取半径不变
        var circle = GetComponent<CircleCollider2D>();
        if (circle != null)
            circle.radius = BaseColliderRadius / factor;
    }

    //  计算精灵"实际内容"的世界尺寸（不透明像素包围盒的长边）。
    //  图集切割帧四周带有透明留白，直接用 ｓｐｒｉｔｅ．ｂｏｕｎｄｓ 会把留白算进去导致武器显示偏小。
    private float GetSpriteContentSize(Sprite sp)
    {
        try
        {
            var tex = sp.texture;
            Rect r = sp.textureRect;
            int ox = (int)r.x, oy = (int)r.y, rw = (int)r.width, rh = (int)r.height;
            Color[] colors = tex.GetPixels(ox, oy, rw, rh);
            int minX = rw, minY = rh, maxX = -1, maxY = -1;
            for (int y = 0; y < rh; y++)
            {
                int row = y * rw;
                for (int x = 0; x < rw; x++)
                {
                    if (colors[row + x].a > 0.1f)
                    {
                        if (x < minX) minX = x;
                        if (x > maxX) maxX = x;
                        if (y < minY) minY = y;
                        if (y > maxY) maxY = y;
                    }
                }
            }
            if (maxX < 0) return Mathf.Max(sp.bounds.size.x, sp.bounds.size.y);
            float worldW = (maxX - minX + 1) / sp.pixelsPerUnit;
            float worldH = (maxY - minY + 1) / sp.pixelsPerUnit;
            return Mathf.Max(worldW, worldH);
        }
        catch (UnityException)
        {
```

Copyright (c) 2026 StarEcoStudio

```
                //  纹理未开启  Ｒｅａｄ/Ｗｒｉｔｅ  时的兜底
            return Mathf.Max(sp.bounds.size.x, sp.bounds.size.y);
        }
    }
    public void Initialize(ItemSO itemSO,int quantity, bool canPickup = true)
    {
        this.itemSO = itemSO;
        this.quantity = quantity;
        this.canPickup = canPickup;
        if (!canPickup)
            playerInside = true;
        UpdateAppearance();
    }
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            if (!canPickup)
                return;
            if (!InventoryManager.Instance.CanAddItem(itemSO))
                return;
                    ｐｉｃｋｅｄＵｐ  =  ｔｒｕｅ；     //  停止逐帧切换，Ｓｐｒｉｔｅ  停在当前帧由  ＬｏｏｔＰｉｃｋｕｐ  动画上浮淡
            anim.Play("LootPickup");
            OnItemLooted?.Invoke(itemSO,quantity);
            Destroy(gameObject,.5f);
        }
    }
    private void OnTriggerExit2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            playerInside = false;
            canPickup = true;
        }
    }
    private void Update()
    {
        if (!canPickup && !playerInside)
        {
            canPickup = true;
        }

        TickWorldFrames();
    }

       //  按配置帧率循环切换精灵帧。位移 / 呼吸由  Ａｎｉｍａｔｏｒ  的  ＬｏｏｔＩｄｌｅ  曲线负责，两者叠加。
    private void TickWorldFrames()
    {
        if (pickedUp || worldFrames == null || worldFrames.Length == 0)
            return;
```

Copyright (c) 2026 StarEcoStudio

```
        frameTimer += Time.deltaTime;
        float interval = 1f / Mathf.Max(1f, worldFrameRate);
        while (frameTimer >= interval)
        {
            frameTimer -= interval;
            frameIndex = (frameIndex + 1) % worldFrames.Length;
            sr.sprite = worldFrames[frameIndex];
        }
    }

}

// ===== File: Assets/Scripts/Inventory & Shop/Shop/ShopButtonToggle.cs =====
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class ShopButtonToggle : MonoBehaviour
{
    public void OpenItemShop()
    {
        if (ShopKeeper.currentShopKeeper != null)
        {
            ShopKeeper.currentShopKeeper.OpenItemShop();
        }
    }
    public void OpenWeaponsShop()
    {
        if (ShopKeeper.currentShopKeeper != null)
        {
            ShopKeeper.currentShopKeeper.OpenWeaponShop();
        }
    }
    public void OpenArmourShop()
    {
        if (ShopKeeper.currentShopKeeper != null)
        {
            ShopKeeper.currentShopKeeper.OpenArmourShop();
        }
    }
}

// ===== File: Assets/Scripts/Inventory & Shop/Shop/ShopCloseButton.cs =====
using UnityEngine;

/// <summary>
/// 商店关闭按钮：点击关闭当前打开的商店。
/// 动态绑定 ShopKeeper.currentShopKeeper，兼容多个商人共享同一 ShopCanvas 的情况
/// （序列化引用只能绑死一个实例，会关错对象的状态）。
public class ShopCloseButton : MonoBehaviour
```

Copyright (c) 2026 StarEcoStudio

```
{
    public void CloseCurrentShop()
    {
        if (ShopKeeper.currentShopKeeper != null)
            ShopKeeper.currentShopKeeper.CloseShop();
    }
}

// ===== File: Assets/Scripts/Inventory & Shop/Shop/ShopInfo.cs =====
using System.Collections;
using System.Collections.Generic;
using TMPro;
using UnityEngine;

public class ShopInfo : MonoBehaviour
{
    public CanvasGroup infoPanel;

    public TMP_Text itemNameText;
    public TMP_Text itemDescriptionText;

    [Header ("Stat Fields")]
    public TMP_Text[] statTexts;

    private RectTransform infoPanelRect;

    private void Awake()
    {
        infoPanelRect=GetComponent<RectTransform>();
    }
    public void ShowItemInfo(ItemSO itemSO)
    {
        infoPanel.alpha = 1;
        itemNameText.text = itemSO.itemName;
        itemDescriptionText.text = itemSO.itemDescription;

        List<string> stats = new List<string>();
        if (itemSO.currentHealth > 0)
            stats.Add("Health:" + itemSO.currentHealth.ToString());
        if (itemSO.damage > 0)
            stats.Add("Damage:" + itemSO.damage.ToString());
        if(itemSO.speed>0)
            stats.Add("Speed:"+itemSO.speed.ToString());
        if(itemSO.duration>0)
            stats.Add("Duration:"+itemSO.duration.ToString());

        if (stats.Count <= 0)
            return;

        for (int i = 0; i < statTexts.Length; i++)
```

Copyright (c) 2026 StarEcoStudio

```
        {
            if (i < stats.Count)
            {
                statTexts[i].text = stats[i];
                statTexts[i].gameObject.SetActive(true);
            }
            else
                statTexts[i].gameObject.SetActive(false);
        }

    }
    public void HideItemInfo()
    {
        infoPanel.alpha = 0;

        itemNameText.text = "";

    }
    public void FollowMouse()
    {
        Vector3 mousePosition = Input.mousePosition;
        Vector3 offest = new Vector3(10, -10, 0);

        infoPanelRect.position = mousePosition + offest;
    }
}

// ===== File: Assets/Scripts/Inventory & Shop/Shop/ShopKeeper.cs =====
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// 商店老板：仅负责商店面板的开关和商品列表。
/// 触发交互（靠近检测 + F 键）由 NPCInteractable 基类接管。
/// 对话 → 选"要买东西" → DialogManager 触发 actionId="openShop" → 调本类 OpenShop(
/// </summary>
public class ShopKeeper : MonoBehaviour
{
    public static ShopKeeper currentShopKeeper;

    public Animator anim;
    public ShopManager shopManager;
    public CanvasGroup shopCanvasGroup;

    [SerializeField] private List<ShopItems> shopItems;
    [SerializeField] private List<ShopItems> shopWeapons;
    [SerializeField] private List<ShopItems> shopArmour;

    [SerializeField] private Camera shopkeeperCam;
```

Copyright (c) 2026 StarEcoStudio

```
    [SerializeField] private Vector3 cameraOffset = new Vector3(0, 0, -1);

    public static event Action<ShopManager, bool> OnShopStateChanged;
    private bool isShopOpen;

       //  NPCInteractable 基类已经有 OnTriggerEnter/Exit，但 ShopKeeper 自己
       //  也想知道玩家在不在（anim 参数），保留一份简单逻辑。
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Player") && anim != null)
            anim.SetBool("playerInRange", true);
    }

    private void OnTriggerExit2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            if (anim != null) anim.SetBool("playerInRange", false);
                    //  玩家离开时强制关闭商店（防止卡住）
            if (isShopOpen) CloseShop();
        }
    }

       ///  <summary>启动时向 DialogManager 注册"开商店"动作。</summary>
    void Start()
    {
        DialogManager.RegisterAction("openShop", OpenShop);
        DialogManager.RegisterAction("closeShop", CloseShop);
    }

    void Update()
    {
             //  Esc 全局关店：timeScale=0 时玩家无法走动触发 OnTriggerExit，
             //  必须提供按键关闭路径（F 关店由 NPCInteractable 处理，这里补 Esc）。
        if (isShopOpen && GameInput.CancelPressed)
        {
            CloseShop();
        }
    }

       ///  <summary>由 DialogManager 在玩家点"要买东西"时触发。</summary>
    public void OpenShop()
    {
        currentShopKeeper = this;
        Time.timeScale = 0;
        isShopOpen = true;
        OnShopStateChanged?.Invoke(shopManager, true);
        shopCanvasGroup.alpha = 1;
        shopCanvasGroup.blocksRaycasts = true;
        shopCanvasGroup.interactable = true;
```

Copyright (c) 2026 StarEcoStudio

```
        [Tooltip("目标类型：击杀敌人 / 收集物品")]
    public QuestObjectiveType type;

        [Tooltip("KillEnemy 填 EnemyHealth.enemyId；CollectItem 填 ItemSO.id")]
    public string targetId;

        [Tooltip("需要的数量")]
    public int requiredCount = 1;

        [Tooltip("任务日志里显示的目标描述（中文），例如：清理火把怪 0/3")]
    public string displayText;
}

/// <summary>
///  任务数据资产。一个资产 = 一个任务。
///  接取/完成的额外条件通过 conditionId 引用 DialogManager 的条件注册表
///  （Func<bool> 委托字典），与对话选项的 conditionId 是同一套机制。
/// </summary>
[CreateAssetMenu(fileName = "NewQuest", menuName = "zeldaLike/Quest")]
public class QuestSO : ScriptableObject
{
        [Header("基础信息")]
        [Tooltip("任务内部标识：全小写英文，全局唯一（如 q_torch_cleanup）")]
    public string questId;

        [Tooltip("给玩家看的任务名（中文）")]
    public string title;

    [TextArea]
        [Tooltip("任务描述（中文），显示在任务日志里")]
    public string description;

        [Header("条件（对应 DialogManager 条件注册表里的 conditionId，留空=无条件）")]
        [Tooltip("接取条件：满足后对话里才会出现接任务选项")]
    public string acceptConditionId;

        [Tooltip("完成条件：所有目标达成后，若此项谓词也为 true 才算 Completed（可用于'还要回去和某人对话'之类）")]
    public string completionConditionId;

        [Header("目标")]
        [Tooltip("结构化进度目标。留空表示纯条件任务（只靠 completionConditionId 判定）")]
    public List<QuestObjective> objectives = new List<QuestObjective>();

        [Header("奖励（交付时发放）")]
        [Tooltip("金币奖励，进 InventoryManager.gold")]
    public int rewardGold;

        [Tooltip("经验奖励，进 ExpManager.GainExperience")]
    public int rewardExp;
}
```

Copyright (c) 2026 StarEcoStudio

```
// ===== File: Assets/Scripts/SkillTree/SkillEffectApplier.cs =====
using UnityEngine;
using System.Collections.Generic;

public class SkillEffectApplier : MonoBehaviour
{
    public PlayerCombat combat;

    private void OnEnable()
    {
        SkillSlot.OnAbilityPointSpent += HandleAbilityPointSpent;
    }
    private void OnDisable()
    {
        SkillSlot.OnAbilityPointSpent -= HandleAbilityPointSpent;
    }
    private void HandleAbilityPointSpent(SkillSlot slot)
    {
        ApplySlotEffect(slot);
    }

    /// <summary>
       /// 施加单个技能槽的**单级**效果。抽成公开方法是为了让调用方能显式驱动效果，
       /// 而不必依赖「花技能点」事件的隐式触发。
    /// </summary>
    public void ApplySlotEffect(SkillSlot slot)
    {
        if (slot == null || slot.skillSO == null) return;

        string skillName = slot.skillSO.skillName;
        switch (skillName)
        {
            case "Max Health Boost":
                StatsManager.Instance.UpdateMaxHealth(1);
                break;
                    // 技能树只负责强化已获得的能力（伤害+1/级）；
                    // 挥剑能力本身只能通过拾取剑道具解锁，不能再用技能点绕过探索
            case "Sword Damage+":
                StatsManager.Instance.playerDamage += 1;
                if (combat != null && combat.statsUI != null) combat.statsUI.UpdateDamage();
                break;

            default:
                Debug.LogWarning("Unknown skill: " + skillName);
                break;
        }
    }
}

using System.Collections;
```

Copyright (c) 2026 StarEcoStudio

```
using System.Collections.Generic;
using System.Net;
using UnityEngine;

[CreateAssetMenu(fileName="NewSkill",menuName="skillTree/skill")]
public class SkillSO: ScriptableObject
{

    public string skillName;
    public int maxLevel;
    public Sprite skillIcon;
}

// ===== File: Assets/Scripts/SkillTree/SkillSlot.cs =====
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using TMPro;
using System;
public class SkillSlot : MonoBehaviour
{
    public SkillSO skillSO;
    public int currentLevel;
    public bool isUnlocked;
    public Image skillIcon;
    public TMP_Text skillLevelText;
    public Button skillButton;
    public static event Action<SkillSlot> OnAbilityPointSpent;
    public static event Action<SkillSlot> OnSkillMaxed;
    public List<SkillSlot> prerequisiteSkillSlots;
    public void TryUpgradeSkill()
    {
        if (skillSO == null) return;

        if (isUnlocked&&currentLevel<skillSO.maxLevel)
        {
            currentLevel++;
            OnAbilityPointSpent?.Invoke(this);
            if (currentLevel >= skillSO.maxLevel)
            {
                OnSkillMaxed?.Invoke(this);

            }
            OnValidate();
        }
    }
    public bool CanUnlocked()
    {
        if (skillSO == null) return false;
```

Copyright (c) 2026 StarEcoStudio

```
        if (prerequisiteSkillSlots == null) return true;

        foreach (SkillSlot slot in prerequisiteSkillSlots)
        {
            if (slot == null || slot.skillSO == null) continue;
            if (!slot.isUnlocked||slot.currentLevel<slot.skillSO.maxLevel )
            {
                return false;
            }
        }
        return true;
    }

    public void Unlock()
    {
        isUnlocked = true;
        UpdateUI();
    }
    private void OnValidate()
    {
        if (skillSO != null&&skillLevelText!=null)
        {

            UpdateUI();
        }
    }
    private void UpdateUI()
    {
        if (skillSO == null || skillIcon == null) return;

        skillIcon.sprite = skillSO.skillIcon;
        if (isUnlocked)
        {
            skillButton.interactable = true;
            skillLevelText.text = currentLevel.ToString()+"/"+skillSO.maxLevel.ToString();
            skillIcon.color=Color.white;
        }
        else
        {
            skillButton.interactable = false;
            skillIcon.color = Color.grey;
            skillLevelText.text = "Locked";
        }
    }

       /// <summary>对外公开的刷新入口：读档直接改字段后调用，避免依赖私有的 UpdateUI。</summary>
    public void RefreshUI()
    {
        UpdateUI();
    }
```

Copyright (c) 2026 StarEcoStudio

```
}

// ===== File: Assets/Scripts/SkillTree/SkillTreeManager.cs =====
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using TMPro;
public class SkillTreeManager : MonoBehaviour
{
    public SkillSlot[] skillSlots;
    public TMP_Text pointsText;
    public int availablePoints;
    void OnEnable()
    {
        SkillSlot.OnAbilityPointSpent += HandleAbilityPointsSpent;
        SkillSlot.OnSkillMaxed += HandleSkillMaxed;
        ExpManager.OnLevelUp += UpdateAbilityPoints;
    }
    void OnDisable()
    {
        SkillSlot.OnAbilityPointSpent -= HandleAbilityPointsSpent;
        SkillSlot.OnSkillMaxed -= HandleSkillMaxed;
        ExpManager.OnLevelUp-=UpdateAbilityPoints;
    }
    void Start()
    {
        foreach (SkillSlot slot in skillSlots)
        {
            if (slot == null || slot.skillButton == null) continue;
            slot.skillButton.onClick.AddListener(() => CheckAvailablePoints(slot));
        }
        UnlockRootSkills();
        UpdateAbilityPoints(0);
    }

    /// <summary>
    /// 无前置的技能是技能树的根，开局即可点。带前置的技能必须靠前置满级来解锁，
    /// 由 <see cref="HandleSkillMaxed" /> 负责。
    /// </summary>
    private void UnlockRootSkills()
    {
        foreach (SkillSlot slot in skillSlots)
        {
            if (slot == null || slot.skillSO == null) continue;
            if (slot.isUnlocked) continue;
            if (slot.prerequisiteSkillSlots != null && slot.prerequisiteSkillSlots.Count > 0) continue;

            slot.Unlock();
        }
    }
```

Copyright (c) 2026 StarEcoStudio

```
    public void CheckAvailablePoints(SkillSlot slot)
    {
        if (availablePoints > 0)
        {
            slot.TryUpgradeSkill();
        }
    }
    private void HandleAbilityPointsSpent(SkillSlot skillSlot)
    {
        if (availablePoints > 0)
        {
            UpdateAbilityPoints(-1);
        }
    }
    /// <summary>
/ / /  某个技能满级后的连锁解锁：
/ / /  只解锁「把本次满级技能列为前置」的槽位。
/ / /  不能遍历全表解锁所有可解锁项——那样任何无前置的根技能都会被其他技能的满级
/ / /  顺带解锁（例如剑伤害满级会误开本应独立的电闪雷鸣）。
/// </summary>
    private void HandleSkillMaxed(SkillSlot skillSlot)
    {
        if (skillSlot == null) return;

        foreach (SkillSlot slot in skillSlots)
        {
            if (slot == null || slot == skillSlot) continue;
            if (slot.isUnlocked) continue;
            if (slot.prerequisiteSkillSlots == null) continue;
            if (!slot.prerequisiteSkillSlots.Contains(skillSlot)) continue;

            if (slot.CanUnlocked())
            {
                slot.Unlock();
            }
        }
    }
    public void UpdateAbilityPoints(int amount)
    {
        availablePoints += amount;
        if (pointsText != null) pointsText.text = "points: " + availablePoints;
    }
}

// ===== File: Assets/Scripts/SkillTree/ToggleSkillTree.cs =====
using System.Collections;
using System.Collections.Generic;
using System.Threading;
using UnityEngine;

public class ToggleSkillTree : MonoBehaviour
```

Copyright (c) 2026 StarEcoStudio

```
{
    public CanvasGroup statsCanvas;
    private bool skillTreeOpen = false;
    public bool IsOpen => skillTreeOpen;

    void Update()
    {
        if (GameInput.ToggleSkillTreePressed)
        {
            if (skillTreeOpen) ClosePanel();
            else OpenPanel();
        }
    }

       // 打开技能面板：先关闭属性面板（互斥），再暂停游戏并显示自己
    public void OpenPanel()
    {
        var stats = FindObjectOfType<StatsUI>();
        if (stats != null && stats.IsOpen) stats.ClosePanel();

        Time.timeScale = 0;
        statsCanvas.alpha = 1;
        statsCanvas.blocksRaycasts = true;
        skillTreeOpen = true;
    }

    public void ClosePanel()
    {
        Time.timeScale = 1;
        statsCanvas.alpha = 0;
        statsCanvas.blocksRaycasts = false;
        skillTreeOpen = false;
    }
}

// ===== File: Assets/Scripts/StateMachine/StateMachine.cs =====
public interface IState<T>
{
    void OnEnter(T host);
    void OnUpdate(T host);
    void OnExit(T host);
}

public class StateMachine<T>
{
    private IState<T> currentState;
    private readonly T host;

    public StateMachine(T host)
    {
```

Copyright (c) 2026 StarEcoStudio

```
        this.host = host;
    }

    public void ChangeState(IState<T> next)
    {
        currentState?.OnExit(host);
        currentState = next;
        currentState.OnEnter(host);
    }

    public void Tick()
    {
        currentState?.OnUpdate(host);
    }
}

// ===== File: Assets/Scripts/UI/FloatingOnScreenStick.cs =====
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.InputSystem.OnScreen;

/// <summary>
///  浮动摇杆：手指在摇杆区任意位置按下时，摇杆在按下处显形并跟随手指滑动，
///  松手隐藏并归零。值写入 controlPath（如 <Gamepad>/leftStick），
///  由 OnScreenControl 自动创建虚拟手柄设备，GameInput.Move 直接可读。
///  注意：OnScreenStick 的指针方法不是 virtual，这里以接口重实现接管事件，
///  仅复用基类的 controlPath、movementRange 与 SendValueToControl。
/// </summary>
public class FloatingOnScreenStick : OnScreenStick,
    IPointerDownHandler, IDragHandler, IPointerUpHandler
{
       [Tooltip("摇杆视觉根（Base+Knob 容器），按下时显形、松手隐藏")]
    public RectTransform visualRoot;

       [Tooltip("摇杆圆钮，跟随手指偏移")]
    public RectTransform knob;

    private int _pointerId = -1;
    private Vector2 _pressScreenPos;

    new public void OnPointerDown(PointerEventData eventData)
    {
            //  已有一根手指控制时忽略多指，防止抢位
        if (_pointerId != -1) return;
        _pointerId = eventData.pointerId;
        _pressScreenPos = eventData.position;

        if (visualRoot != null)
        {
            visualRoot.gameObject.SetActive(true);
```

Copyright (c) 2026 StarEcoStudio

```
            if (RectTransformUtility.ScreenPointToLocalPointInRectangle(
                    (RectTransform)transform, eventData.position, eventData.pressEventCamera, out var local))
            {
                visualRoot.localPosition = local;
            }
        }
        if (knob != null) knob.anchoredPosition = Vector2.zero;
        SendValueToControl(Vector2.zero);
    }

    new public void OnDrag(PointerEventData eventData)
    {
        if (eventData.pointerId != _pointerId) return;

            // movementRange 是 Canvas 本地单位，屏幕位移要除以缩放
        var canvas = GetComponentInParent<Canvas>();
        float scale = canvas != null ? canvas.scaleFactor : 1f;
        Vector2 delta = Vector2.ClampMagnitude(
            (eventData.position - _pressScreenPos) / scale, movementRange);

        if (knob != null) knob.anchoredPosition = delta;
        SendValueToControl(delta / movementRange);
    }

    new public void OnPointerUp(PointerEventData eventData)
    {
        if (eventData.pointerId != _pointerId) return;
        ResetStick();
        SendValueToControl(Vector2.zero);
    }

    protected override void OnDisable()
    {
        ResetStick();
        base.OnDisable();
    }

    private void ResetStick()
    {
        _pointerId = -1;
        if (knob != null) knob.anchoredPosition = Vector2.zero;
        if (visualRoot != null) visualRoot.gameObject.SetActive(false);
    }
}

// ===== File: Assets/Scripts/UI/MenuHubUI.cs =====
using System;
using TMPro;
using UnityEngine;

public class MenuHubUI : MonoBehaviour
```

Copyright (c) 2026 StarEcoStudio

```
{
       [Ｔｏｏｌｔｉｐ("中枢面板的 ＣａｎｖａｓＧｒｏｕｐ")]
    public CanvasGroup hubCanvas;

    public StatsUI statsUI;
    public ToggleSkillTree skillTreeUI;
    public QuestLogUI questLogUI;

    private bool _open;
    public bool IsOpen => _open;

    void Start()
    {
        CloseHub();
    }

    public void ToggleHub()
    {
        if (_open) CloseHub();
        else OpenHub();
    }

    public void OpenHub()
    {
        if (DialogManager.Instance != null && DialogManager.Instance.IsInDialog) return;
        if (hubCanvas == null) return;
        hubCanvas.alpha = 1;
        hubCanvas.blocksRaycasts = true;
        hubCanvas.interactable = true;
        _open = true;
    }

    public void CloseHub()
    {
        if (hubCanvas == null) return;
        hubCanvas.alpha = 0;
        hubCanvas.blocksRaycasts = false;
        hubCanvas.interactable = false;
        _open = false;
    }

       // 三个条目按钮的 ＯｎＣｌｉｃｋ 绑定
    public void OpenStats()
    {
        CloseHub();
        if (statsUI != null) statsUI.OpenPanel();
    }

    public void OpenSkillTree()
    {
```

Copyright (c) 2026 StarEcoStudio

```
        CloseHub();
        if (skillTreeUI != null) skillTreeUI.OpenPanel();
    }

    public void OpenQuestLog()
    {
        CloseHub();
        if (questLogUI != null) questLogUI.OpenPanel();
    }

    // ========== 存档条目 ==========

    [Tooltip("手动存档按钮（留空则按钮仍可用，只是不提示路径）")]
    public TMP_Text saveHint;

    public void SaveGame()
    {
        CloseHub();
        if (SaveSystem.Instance == null)
        {
            Debug.LogWarning("[MenuHubUI] SaveSystem 未就绪，无法存档");
            return;
        }
        if (SaveSystem.Instance.SaveGame() && saveHint != null)
            saveHint.text = "已存档 " + DateTime.Now.ToString("HH:mm:ss");
    }

    public void LoadGame()
    {
        CloseHub();
        if (SaveSystem.Instance == null)
        {
            Debug.LogWarning("[MenuHubUI] SaveSystem 未就绪，无法读档");
            return;
        }
        if (!SaveSystem.HasSave)
        {
            Debug.LogWarning("[MenuHubUI] 还没有存档");
            if (saveHint != null) saveHint.text = "暂无存档";
            return;
        }
        SaveSystem.Instance.LoadGame();
    }
}

// ===== File: Assets/Scripts/UI/SafeAreaFitter.cs =====
using UnityEngine;

/// <summary>
/// 把 Screen.safeArea 应用到自身 RectTransform 的锚点，
public class SafeAreaFitter : MonoBehaviour
```

Copyright (c) 2026 StarEcoStudio

```
{
       [Ｔｏｏｌｔｉｐ("仅编辑器生效：用模拟安全区预览刘海内缩效果（真机自动使用真实安全区）")]
    public bool simulateInEditor;

       [Ｔｏｏｌｔｉｐ("模拟安全区（０~１ 归一化矩形：ｘ/ｙ/ｗｉｄｔｈ/ｈｅｉｇｈｔ），仅编辑器生效")]
    public Rect simulatedSafeArea = new Rect(0f, 0.07f, 1f, 0.86f);

    private RectTransform _rt;
    private Rect _lastSafeArea = new Rect(-1f, -1f, -1f, -1f);
    private int _lastWidth, _lastHeight;

    void Awake()
    {
        _rt = (RectTransform)transform;
        Apply();
    }

    void Update()
    {
        if (_lastSafeArea != CurrentSafeArea() ||
            _lastWidth != Screen.width || _lastHeight != Screen.height)
        {
            Apply();
        }
    }

    private Rect CurrentSafeArea()
    {
#if UNITY_EDITOR
        if (simulateInEditor)
        {
            float w = Screen.width, h = Screen.height;
            return new Rect(
                simulatedSafeArea.x * w,
                simulatedSafeArea.y * h,
                simulatedSafeArea.width * w,
                simulatedSafeArea.height * h);
        }
#endif
        return Screen.safeArea;
    }

    private void Apply()
    {
        if (_rt == null) return;

        Rect sa = CurrentSafeArea();
        _lastSafeArea = sa;
        _lastWidth = Screen.width;
        _lastHeight = Screen.height;
```

Copyright (c) 2026 StarEcoStudio

```
        Vector2 min = sa.position;
        Vector2 max = sa.position + sa.size;
        min.x /= Screen.width;
        min.y /= Screen.height;
        max.x /= Screen.width;
        max.y /= Screen.height;

        _rt.anchorMin = min;
        _rt.anchorMax = max;
        _rt.offsetMin = Vector2.zero;
        _rt.offsetMax = Vector2.zero;
    }
}

// ===== File: Assets/Scripts/UI/TouchUIController.cs =====
using UnityEngine;
using UnityEngine.InputSystem;

/// <summary>
/// 触屏控件显隐门控：
/// 1） 总开关——手机平台默认开启，PC/编辑器默认关闭（forceShow 可强制预览）；
/// 2） 游戏进行中才显示——timeScale=0（面板/商店/对话暂停）或对话进行中一律隐藏，
///       防止暂停期摇杆残留移动输入、跨 Canvas 射线穿透；
/// 3） 检测到实体手柄时隐藏，避免虚拟手柄与实体手柄输入打架。
/// </summary>
public class TouchUIController : MonoBehaviour
{
       [Tooltip("触屏控件根（摇杆区+按钮簇+菜单按钮）")]
    public GameObject controlsRoot;

       [Tooltip("PC/编辑器强制显示触屏控件（预览测试用）")]
    public bool forceShow;

    void Update()
    {
        bool should = ShouldShow();
        if (controlsRoot != null && controlsRoot.activeSelf != should)
            controlsRoot.SetActive(should);
    }

    private bool ShouldShow()
    {
        if (!forceShow && !Application.isMobilePlatform) return false;
        if (HasPhysicalGamepad()) return false;
        if (Time.timeScale <= 0f) return false;
        if (DialogManager.Instance != null && DialogManager.Instance.IsInDialog) return false;
        return true;
    }

    private static bool HasPhysicalGamepad()
```

Copyright (c) 2026 StarEcoStudio

```
    {
        foreach (var pad in Gamepad.all)
        {
            bool onScreen = false;
            foreach (var usage in pad.usages)
            {
                if (usage.ToString() == "OnScreen")
                {
                    onScreen = true;
                    break;
                }
            }
            if (!onScreen) return true;
        }
        return false;
    }
}

// ===== File: Assets/Scripts/AreaGate.cs =====
using System.Collections;
using UnityEngine;
using UnityEngine.SceneManagement;

/// <summary>
///  区域门禁：玩家踩上触发器后，按条件决定放行切场景还是提示。
///  每个门禁配置目标场景与到达后的玩家出生点，出生点经静态字段跨场景传递。
/// </summary>
public class AreaGate : MonoBehaviour
{
    public enum GateCondition { None, HasSword }

    [Header("Gate Settings")]
       [Tooltip("通过门禁需要满足的条件；None 表示自由通行（用于反向传送点）")]
    public GateCondition condition = GateCondition.None;

       [Tooltip("通过门禁后加载的目标场景名")]
    public string targetSceneName;

       [Tooltip("进入目标场景后玩家的出生位置")]
    public Vector2 targetSpawnPosition;

    [Header("Locked Hint")]
       [Tooltip("未满足条件时显示的提示文本（TextMesh，中文用内置 Arial 动态字体），留空则无提示")]
    public TextMesh lockedHint;

       [Tooltip("提示文本显示时长（秒）")]
    public float hintDuration = 2.5f;

       // 跨场景传递出生点：目标场景里的 PlayerSpawn 在 Start 时消费
    static bool _hasPendingSpawn;
```

Copyright (c) 2026 StarEcoStudio

```
    static Vector3 _pendingSpawnPosition;

    public static bool HasPendingSpawn => _hasPendingSpawn;

    public static Vector3 ConsumePendingSpawn()
    {
        _hasPendingSpawn = false;
        return _pendingSpawnPosition;
    }

    Coroutine _hintRoutine;

    void OnTriggerEnter2D(Collider2D other)
    {
        if (!other.CompareTag("Player")) return;

        if (condition == GateCondition.HasSword && !StatsManager.Instance.hasSword)
        {
            ShowLockedHint();
            return;
        }

        _pendingSpawnPosition = targetSpawnPosition;
        _hasPendingSpawn = true;
        SceneManager.LoadScene(targetSceneName);
    }

    void ShowLockedHint()
    {
        if (lockedHint == null) return;
        lockedHint.gameObject.SetActive(true);
        if (_hintRoutine != null) StopCoroutine(_hintRoutine);
        _hintRoutine = StartCoroutine(HideHintRoutine());
    }

    IEnumerator HideHintRoutine()
    {
        yield return new WaitForSeconds(hintDuration);
        lockedHint.gameObject.SetActive(false);
    }
}

// ===== File: Assets/Scripts/CameraFollow.cs =====
using UnityEngine;

/// <summary>
/// 轻量相机跟随：ForestArea 没有搭 Cinemachine，用硬跟随保持玩家居中。
/// </summary>
public class CameraFollow : MonoBehaviour
{
```

Copyright (c) 2026 StarEcoStudio

```
    public Transform target;

    void Start()
    {
            //  无条件绑定到当前场景中"活跃"的 Ｐｌａｙｅｒ：切场景回来时序列化 ｔａｒｇｅｔ 指向的
            //  重复 Ｐｌａｙｅｒ 此时只是停用、帧末才销毁，判空不可靠；ＦｉｎｄＧａｍｅＯｂｊｅｃｔＷｉｔｈＴａｇ
            //  只返回活跃对象，天然指向持久存活的那个
        var player = GameObject.FindGameObjectWithTag("Player");
        if (player != null) target = player.transform;
    }

    void LateUpdate()
    {
        if (target == null) return;
        transform.position = new Vector3(target.position.x, target.position.y, transform.position.z);
    }
}

// ===== File: Assets/Scripts/CameraTargetBinder.cs =====
using UnityEngine;

/// <summary>
///  场景相机绑定器：场景加载后把虚拟相机的 Ｆｏｌｌｏｗ/ＬｏｏｋＡｔ 绑定到持久存活的 Ｐｌａｙｅｒ。
///  场景里的相机不放进 ＧａｍｅＲｏｏｔ（每个场景构图/Ｃｏｎｆｉｎｅｒ 不同），切场景回来时
///  新相机序列化的 Ｆｏｌｌｏｗ 可能指向已被 ＧａｍｅＢｏｏｔｓｔｒａｐ 守卫销毁的重复 Ｐｌａｙｅｒ，
///  这里检测到空引用就用当前存活的 Ｐｌａｙｅｒ 覆盖。
///  Ｃｉｎｅｍａｃｈｉｎｅ 程序集未被项目脚本直接引用，用反射设置其 Ｆｏｌｌｏｗ/ＬｏｏｋＡｔ 属性。
/// </summary>
public class CameraTargetBinder : MonoBehaviour
{
    void Start()
    {
            //  无条件绑定到活跃 Ｐｌａｙｅｒ：重复 ＧａｍｅＲｏｏｔ 在 Ａｗａｋｅ 被停用、帧末才销毁，
            //  此时旧序列化引用判空不可靠，而 ＦｉｎｄＧａｍｅＯｂｊｅｃｔＷｉｔｈＴａｇ 只返回活跃对象，
            //  天然得到持久存活的那个
        var player = GameObject.FindGameObjectWithTag("Player");
        if (player == null) return;

        foreach (var c in GetComponents<Component>())
        {
            if (c == null || c.GetType().Name != "CinemachineVirtualCamera") continue;

            var followProp = c.GetType().GetProperty("Follow");
            if (followProp != null)
                followProp.SetValue(c, player.transform);
                    //  不设 ＬｏｏｋＡｔ：２Ｄ 相机管线无 Ａｉｍ 阶段（Ｆｒａｍｉｎｇ Ｔｒａｎｓｐｏｓｅｒ 只用 Ｆｏｌｌｏｗ），
                    //  旋转必须保持固定，否则 Ｃｏｍｐｏｓｅｒ 式瞄准会持续微转镜头产生画面摇摆
        }
    }
}
```

Copyright (c) 2026 StarEcoStudio

```
// ===== File: Assets/Scripts/ElevationEntry.cs =====
using UnityEngine;

public class ElevationEntry : MonoBehaviour
{
    public Collider2D[] mountainColliders;
    public Collider2D[] mountainBoundary;
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.gameObject.tag == "Player")
        {
            collision.gameObject.GetComponent<SpriteRenderer>().sortingOrder = 15;
            foreach (Collider2D mountain in mountainColliders)
            {
                mountain.enabled = false;
            }
            foreach (Collider2D boundary in mountainBoundary)
            {
                boundary.enabled = true;
            }
        }
    }
}

// ===== File: Assets/Scripts/ElevationExit.cs =====
using UnityEngine;

public class ElevationExit : MonoBehaviour
{
    [UnityEngine.Serialization.FormerlySerializedAs("montainBody")]
    public Collider2D[] mountainBody;
    [UnityEngine.Serialization.FormerlySerializedAs("montainBoundary")]
    public Collider2D[] mountainBoundary;

    void OnTriggerEnter2D(Collider2D co)
    {
        if (co.gameObject.tag == "Player")
        {
            co.gameObject.GetComponent<SpriteRenderer>().sortingOrder = 5;
            foreach (Collider2D body in mountainBody)
            {

                body.enabled = true;

            }
            foreach (Collider2D boundary in mountainBoundary)
            {
                boundary.enabled = false;
            }
```

Copyright (c) 2026 StarEcoStudio

```
        }
    }
}

// ===== File: Assets/Scripts/GameBootstrap.cs =====
using UnityEngine;

/// <summary>
///  持久根守卫：整个游戏只允许一个 GameRoot 存活（社区主流的 Bootstrap Prefab 单例模式）。
///  每个场景都放一个 GameRoot 预制体实例（内含 Player、数据管理器与 HUD）。
///  切场景后新场景的重复实例在 Awake 立即整体停用并销毁：
///  - SetActive(false) 在父节点 Awake 中执行，子节点的 Awake/OnEnable 不会再跑，
///    避免重复单例、重复 EventSystem、重复玩家在首帧产生副作用；
///  - 旧实例通过 DontDestroyOnLoad 跨场景存活，血量/金币/背包/经验/能力全部保留。
/// </summary>
public class GameBootstrap : MonoBehaviour
{
    public static GameBootstrap Instance { get; private set; }

    void Awake()
    {
        if (Instance != null && Instance != this)
        {
            gameObject.SetActive(false);
            Destroy(gameObject);
            return;
        }

        Instance = this;
        DontDestroyOnLoad(gameObject);
    }
}

// ===== File: Assets/Scripts/InventoryManager.cs =====
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using TMPro;

public class InventoryManager : MonoBehaviour
{
    public static InventoryManager Instance;
    public InventorySlot[] itemSlots;
    [UnityEngine.Serialization.FormerlySerializedAs("useItem")]
    public ItemEffectApplier itemEffectApplier;
    public int gold;
    public TMP_Text goldText;
    public GameObject lootPrefab;
    public Transform player;
    private void Awake()
```

Copyright (c) 2026 StarEcoStudio

```
    {
        if (Instance == null)
            Instance = this;
        else if (Instance != this)
            Destroy(this);
        goldText.text = gold.ToString();
    }
    private void OnEnable()
    {
        Loot.OnItemLooted += AddItem;
    }
    private void OnDisable()
    {
        Loot.OnItemLooted -= AddItem;
    }
    private void DropLoot(ItemSO itemSO,int quantity, bool canPickup = true)
    {
        Loot loot = Instantiate(lootPrefab, player.position, Quaternion.identity).GetComponent<Loot>();
        loot.Initialize(itemSO, quantity, canPickup);
    }
    public void DropItem(InventorySlot slot)
    {
        if (slot.itemSO == null || slot.quantity <= 0)
            return;

              // 能力道具（剑/弓）永久持有，禁止丢弃——对齐塞尔达道具栏行为
        if (slot.itemSO.unlocksAbility != Ability.None)
            return;

        DropLoot(slot.itemSO, 1, false);
        slot.quantity--;
        if (slot.quantity <= 0)
        {
            slot.itemSO = null;
        }
        slot.UpdateUI();
    }
    public bool CanAddItem(ItemSO itemSO)
    {
        if (itemSO.isGold)
            return true;
        bool hasSlot = false;
        foreach (var slot in itemSlots)
        {
            if (slot.itemSO == itemSO)
            {
                hasSlot = true;
                if (slot.quantity < itemSO.stackSize)
                    return true;
            }
```

Copyright (c) 2026 StarEcoStudio

```
        }
        if (hasSlot)
            return false;
        foreach (var slot in itemSlots)
        {
            if (slot.itemSO == null)
                return true;
        }
        return false;
    }
       ///  <summary>增加金币并刷新 UI。任务奖励等系统统一走这里。</summary>
    public void AddGold(int amount)
    {
        if (amount <= 0) return;
        gold += amount;
        if (goldText != null) goldText.text = gold.ToString();
    }
    public void AddItem(ItemSO itemSO,int quantity)
    {
        if (itemSO.isGold)
        {
            AddGold(quantity);
            return;
        }

              // 塞尔达式设计：能力道具碰到即解锁，无需再手动使用
        if (itemSO.unlocksAbility != Ability.None)
        {
            itemEffectApplier.ApplyItemEffects(itemSO);
        }

        foreach (var slot in itemSlots)
        {
            if (slot.itemSO == itemSO && slot.quantity < itemSO.stackSize)
            {
                int availableSpace = itemSO.stackSize - slot.quantity;
                int amountToAdd = Mathf.Min(availableSpace, quantity);
                slot.quantity += amountToAdd;
                quantity -=amountToAdd;

                slot.UpdateUI();
                if (quantity <= 0)
                {
                    return;
                }
                DropLoot( itemSO,quantity);
                return;

            }
        }
```

Copyright (c) 2026 StarEcoStudio

```
                foreach (var slot in itemSlots)
                {
                    if (slot.itemSO == null)
                    {
                        int amountToAdd = Mathf.Min(itemSO.stackSize, quantity);
                        slot.itemSO = itemSO;
                        slot.quantity = amountToAdd;
                        quantity -= amountToAdd;
                        slot.UpdateUI();
                        if (quantity <= 0)
                            return;
                        DropLoot(itemSO, quantity);
                        return;
                    }

                }




    }
    public void UseItem(InventorySlot slot)
    {
        if (slot.itemSO != null && slot.quantity >= 0)
        {
            itemEffectApplier.ApplyItemEffects(slot.itemSO);

                    // 能力道具（剑/弓）永久持有，使用不消耗数量——对齐塞尔达正版的道具栏行为
            if (slot.itemSO.unlocksAbility != Ability.None)
            {
                return;
            }

            slot.quantity--;
            if (slot.quantity <= 0)
            {
                slot.itemSO = null;
            }
            slot.UpdateUI();

        }
    }
}

// ===== File: Assets/Scripts/InventorySlot.cs =====
using System.Collections;
using System.Collections.Generic;
```

Copyright (c) 2026 StarEcoStudio

```
using UnityEngine;
using UnityEngine.UI;
using TMPro;
using UnityEngine.EventSystems;

public class InventorySlot : MonoBehaviour,IPointerClickHandler
{
    public ItemSO itemSO;
    public int quantity;

    public Image itemImage;
    public TMP_Text quantityText;

    private InventoryManager inventoryManager;
    private static ShopManager activeShop;



    public void Start()
    {
        inventoryManager = GetComponentInParent<InventoryManager>();
    }
    private void OnEnable()
    {
        ShopKeeper.OnShopStateChanged += HandleShopStateChanged;
    }
    private void OnDisable()
    {
        ShopKeeper.OnShopStateChanged -= HandleShopStateChanged;
    }
    private void HandleShopStateChanged(ShopManager shopManager,bool isOpen)

    {
        activeShop = isOpen ? shopManager : null;
    }
    public void OnPointerClick(PointerEventData eventData)
    {
        if (quantity > 0)
        {
             if (eventData.button == PointerEventData.InputButton.Left)
            {
                if (activeShop != null)
                {
                                    // 能力道具为永久持有，不可出售
                    if (itemSO.unlocksAbility != Ability.None) return;
                    activeShop.SellItem(itemSO);
                    quantity--;
                    UpdateUI();
                }
                else
                {
```

Copyright (c) 2026 StarEcoStudio

```
                                    //  能力道具获得即永久生效，无需在背包中手动"使用"
                    if (itemSO.unlocksAbility != Ability.None) return;
                    inventoryManager.UseItem(this);
                }
            }
            else if (eventData.button == PointerEventData.InputButton.Right)
            {
                inventoryManager.DropItem(this);
            }
        }
    }


    public void UpdateUI()
    {
        if (quantity <= 0)
        {
            itemSO = null;
        }
        if (itemImage == null || quantityText == null) return;
        if (itemSO != null)
        {
            itemImage.gameObject.SetActive(true);
            itemImage.sprite = itemSO.icon;
            itemImage.color = new Color(1f, 1f, 1f, 1f);
                    //  只要槽内有物品就显示当前数量（含 1 个）；空格在下方分支清空
            quantityText.text = quantity.ToString();
        }
        else
        {
            itemImage.gameObject.SetActive(false);
            quantityText.text = "";
        }
    }
}

// ===== File: Assets/Scripts/ItemEffectApplier.cs =====
using UnityEngine;

public class ItemEffectApplier : MonoBehaviour
{
    private PlayerController playerController;

    private void Awake()
    {
        playerController = FindObjectOfType<PlayerController>();
    }

    public void ApplyItemEffects(ItemSO item)
    {
```

Copyright (c) 2026 StarEcoStudio

```
        if (item.currentHealth > 0)
        {
            StatsManager.Instance.UpdateHealth(item.currentHealth);
        }
        if (item.maxHealth > 0)
        {
            StatsManager.Instance.UpdateMaxHealth(item.maxHealth);
        }
        if (item.speed > 0)
        {
            StatsManager.Instance.UpdateSpeed(item.speed);
        }
        if (item.duration > 0)
        {
            StartCoroutine(EffectTimer(item, item.duration));
        }

              //  能力解锁是永久的，不走 EffectTimer 的临时 buff 回滚路径
        ApplyAbilityUnlock(item);
    }

    private void ApplyAbilityUnlock(ItemSO item)
    {
        switch (item.unlocksAbility)
        {
            case Ability.Sword:
                StatsManager.Instance.hasSword = true;
                break;
            case Ability.Bow:
                StatsManager.Instance.hasBow = true;
                break;
            case Ability.None:
            default:
                return;
        }

              //  通知玩家控制器按新解锁的能力启用对应战斗组件
        if (playerController != null)
        {
            playerController.RefreshUnlockedAbilities();
        }
    }

    private System.Collections.IEnumerator EffectTimer(ItemSO item, float duration)
    {
        yield return new UnityEngine.WaitForSeconds(duration);
        if (item.currentHealth > 0)
        {
            StatsManager.Instance.UpdateHealth(-item.currentHealth);
        }
```

Copyright (c) 2026 StarEcoStudio

```
        if (item.maxHealth > 0)
        {
            StatsManager.Instance.UpdateMaxHealth(-item.maxHealth);
        }
        if (item.speed > 0)
        {
            StatsManager.Instance.UpdateSpeed(-item.speed);
        }
    }
}

// ===== File: Assets/Scripts/MobileRuntime.cs =====
using UnityEngine;

/// <summary>
/// 手游运行时适配（挂在 GameRoot）：
///
/// 1. **切后台暂停** —— 手机上切出去（来电、微信、切任务）Unity 收到
///       OnApplicationPause(true)。若不暂停，回来时游戏会「自己往前跑」，
///       玩家可能因为操作延迟被怪打死；而低端机切后台再回来常被系统直接回收进程，
///       没存档就丢进度。所以切后台必须：暂停 + 自动存档。
///
/// 2. **自动存档** —— 与 SaveSystem 的退出自动存档配合，构成
///       「切后台 / 退出都存一次」的双保险。
///
/// 3. **帧率与休眠** —— 移动端统一 60 帧上限、禁止自动熄屏（战斗中被锁屏很糟）。
///       帧率走代码而非 PlayerSettings，是因为编辑器与各平台 QualitySettings 的
///       vSync 行为不一致，代码里设一次最可靠。
///
/// 4. **多点触控** —— 明确请求多点支持，否则部分机型只上报第一个触点，
///       摇杆 + 攻击键同时按会失效。
/// </summary>
public class MobileRuntime : MonoBehaviour
{
       [Tooltip("切后台时是否自动存档（强烈建议保持开启）")]
    public bool autoSaveOnPause = true;

       [Tooltip("移动端目标帧率")]
    public int targetFrameRate = 60;

       [Tooltip("是否禁止自动熄屏")]
    public bool preventSleep = true;

    private bool _wasPlayingBeforePause;

    void Awake()
    {
        ApplyRuntimeSettings();
    }

    private void ApplyRuntimeSettings()
```

Copyright (c) 2026 StarEcoStudio

```
    {
        Application.targetFrameRate = targetFrameRate > 0 ? targetFrameRate : 60;
        QualitySettings.vSyncCount = 0;

        if (preventSleep) Screen.sleepTimeout = SleepTimeout.NeverSleep;

              // 移动端多点触控：摇杆与动作键需要能同时响应
        Input.multiTouchEnabled = true;

              // runInBackground=false：切到后台就停止渲染，省电且避免系统回收进程
        Application.runInBackground = false;
    }

    private void OnApplicationPause(bool pauseStatus)
    {
              // 暂停瞬间记录，恢复时才知道该不该解除暂停
        if (pauseStatus)
        {
            _wasPlayingBeforePause = !IsAnyPanelPaused();
        }

        if (!pauseStatus)
        {
                    // 恢复：只解除「切后台造成的」暂停，不能踩掉玩家自己打开的面板
            if (_wasPlayingBeforePause) Time.timeScale = 1f;
            return;
        }

        if (autoSaveOnPause && SaveSystem.Instance != null)
        {
            SaveSystem.Instance.SaveGame();
        }
    }

    private void OnApplicationFocus(bool hasFocus)
    {
              // 部分安卓机型（分屏/画中画）只触发 Focus 而不触发 Pause
        if (hasFocus) return;
        if (autoSaveOnPause && SaveSystem.Instance != null)
        {
            SaveSystem.Instance.SaveGame();
        }
    }

       /// <summary>当前是否已被某个 UI 面板暂停（避免恢复时误把面板关掉）。</summary>
    private static bool IsAnyPanelPaused()
    {
        if (DialogManager.Instance != null && DialogManager.Instance.IsInDialog) return true;
        return Time.timeScale <= 0f;
    }
```

Copyright (c) 2026 StarEcoStudio

```
}

// ===== File: Assets/Scripts/StatsManager.cs =====
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using TMPro;

public class StatsManager : MonoBehaviour
{
    public static  StatsManager Instance;
    public StatsUI statsUI;
    public TMP_Text playerHealthText;

    [Header("Combat Stats")]
    public float playerweaponRange;
    public float playerKnockbackForce;
    public float playerKnockbackStun;
    public int playerDamage;



    [Header("Movement Stats")]
    public int playerSpeed = 5;

    [Header("Health Stats")]
    public float enemyMaxHealth;
    public float enemyCurrentHealth;
    public float playerMaxHealth;
    public float playerCurrentHealth;

       // Unlocked Abilities：能力解锁是永久的，区别于 ItemEffectApplier 里的 duration 临时
       // 静态后备字段让能力跨场景切换不丢失（属性不进 Inspector，状态以代码为准）
    public bool hasSword
    {
        get => _hasSword;
        set => _hasSword = value;
    }
    public bool hasBow
    {
        get => _hasBow;
        set => _hasBow = value;
    }

       // 静态后备字段：每个场景各有一份 StatsManager 实例，但能力状态全局唯一
    static bool _hasSword = false;
    static bool _hasBow = false;

    private void Awake()
    {
        if (Instance == null)
```

Copyright (c) 2026 StarEcoStudio

```
        {
            Instance=this;
        }
        else if(Instance!=null&&Instance!=this)
        {
            Destroy(this);
        }
    }
    public void UpdateMaxHealth(int amount)
    {
        playerMaxHealth += amount;
        playerHealthText.text = "HP:" + playerCurrentHealth + "/" + playerMaxHealth;

    }
    public void UpdateHealth(int amount)
    {
        playerCurrentHealth += amount;
        if (playerCurrentHealth < enemyMaxHealth)
        {

            playerHealthText.text = "HP:" + playerCurrentHealth + "/" + playerMaxHealth;
        }
        else
        {
            playerCurrentHealth = playerMaxHealth;
            playerHealthText.text = "HP:" + playerCurrentHealth + "/" + playerMaxHealth;

        }
    }
    public void UpdateSpeed(int amount)
    {
        playerSpeed += amount;
        statsUI.UpdateAllStats();


    }
}

// ===== File: Assets/Scripts/StatsUI.cs =====
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using TMPro;
using System.Threading;
using System.Security.AccessControl;

public class StatsUI : MonoBehaviour
{
    public GameObject[] statsSlots;
    public CanvasGroup statsCanvas;
```

Copyright (c) 2026 StarEcoStudio

```
    private bool statsOpen = false;
    public bool IsOpen => statsOpen;

    private void Start()
    {
      UpdateAllStats();
    }
    private void Update()
    {
        if (GameInput.ToggleStatsPressed)
        {
            if (statsOpen) ClosePanel();
            else OpenPanel();
        }
    }

       // 打开属性面板：先关闭技能面板（互斥），再暂停游戏并显示自己。
       // 暂停统一使用 timeScale（与技能面板一致），不再分别禁用各组件。
    public void OpenPanel()
    {
        var skillTree = FindObjectOfType<ToggleSkillTree>();
        if (skillTree != null && skillTree.IsOpen) skillTree.ClosePanel();

        Time.timeScale = 0;
        statsCanvas.alpha = 1;
        statsCanvas.blocksRaycasts = true;
        statsOpen = true;
        UpdateAllStats();
    }

    public void ClosePanel()
    {
        statsCanvas.alpha = 0;
        statsCanvas.blocksRaycasts = false;
        statsOpen = false;
              // 两面板互斥：关闭时另一个必然也关着，可以安全恢复时间
        Time.timeScale = 1;
    }
    public void UpdateDamage()
    {
        if (statsSlots == null || statsSlots.Length < 1) return;
        statsSlots[0].GetComponentInChildren<TMP_Text>().text = "Damage: " + StatsManager.Instance.playerDamage.
    }
    public void UpdateSpeed()
    {
        if (statsSlots == null || statsSlots.Length < 2) return;
        statsSlots[1].GetComponentInChildren<TMP_Text>().text = "Speed: " + StatsManager.Instance.playerSpeed.To
    }
    public void UpdateAllStats()
    {
```

Copyright (c) 2026 StarEcoStudio

```
        UpdateDamage();
        UpdateSpeed();
    }
}
```

Copyright (c) 2026 StarEcoStudio